

续表

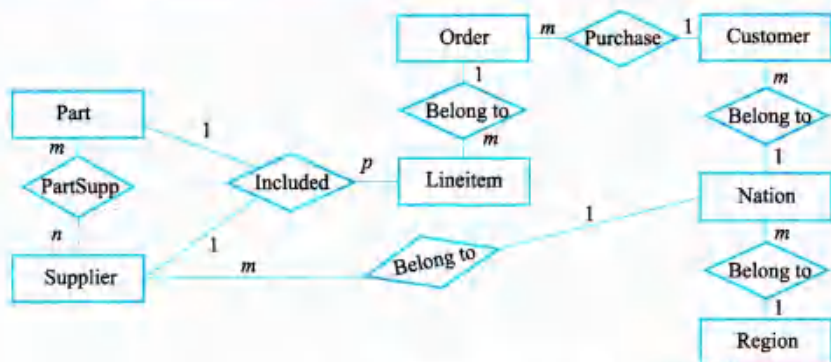


图2 零件供应销售数据库的 E-R 图

(2) 数据库逻辑结构设计

按照数据库设计原理中概念结构转化成逻辑结构的规则，每个实体转换成一个关系，多对多的联系也转换成一个关系。因此，根据上述 E-R 图设计数据库逻辑结构（参见实验 3.1 数据库定义中的图 1）。

(3) 数据库物理结构设计

数据库物理结构首先根据逻辑结构自动转换生成，然后根据应用需求设计数据库的索引结构、存储结构。

(4) 数据库模式 SQL 语句生成

生成的数据库模式 SQL 语句参见实验 3.1 数据库定义的 SQL 语句。

实验总结：

数据库设计的三个主要阶段是概念、逻辑和物理三种结构设计，它们各有不同的设计方法和内容。关键是在概念结构设计中要正确识别实体及其属性，并用合适的名字给实体和属性命名，然后画出符合应用语义的 E-R 图；在逻辑结构设计中要为属性选用合适的数据类型，切忌不管实际情况如何，将所有属性都设为 CHAR 类型，长度一律为 20 或 100 等；在物理结构设计阶段主要是设计适当的索引，为表、索引、日志等分配合理的存储区。

3. 思考题

- (1) 请选择一个熟悉的应用，练习数据库设计。
- (2) 请使用 PowerDesigner 等数据库辅助设计工具软件进行数据库设计。

第 8 章实验 数据库编程与大作业

数据库编程与大作业实验包含 5 个实验项目（见表 8），其中 2 个必修实验项目，3 个选修

实验项目，它们均为设计性实验。

表8 “数据库编程与大作业”实验项目一览表

序号	实验项目名称	开设类别	难易程度	建议实验学时	对应《概论》章节
实验 8.1	存储过程实验	必修	较难	2	第8章 8.2.5 小节
实验 8.2	自定义函数实验	选修	较难	2	第8章 8.2.6 小节
实验 8.3	游标实验	选修	较难	2	第8章 8.2.4 小节
实验 8.4	基于 JDBC 的数据库应用开发实验	选修	较难	4	第8章 8.3、8.4 节
实验 8.5	数据库大作业	必修	较难	8	第6章~第8章

实验 8.1 存储过程实验

1. 实验内容和要求

掌握数据库编程语言 PL/SQL 以及数据库存储过程的设计和使用方法，包括存储过程定义、存储过程运行，存储过程更名、存储过程删除、存储过程的参数传递；掌握 PL/SQL 编程规范，规范设计存储过程。

实验重点：存储过程的定义和运行。

实验难点：存储过程的参数传递方法。

2. 实验报告示例

实验报告			
题目：存储过程实验	姓名	日期	
(1) 无参数的存储过程 ① 定义一个存储过程，更新所有订单的（含税折扣价）总价。 <pre> /*即根据订单明细表，计算每个订单的总价，更新 Orders 表*/ CREATE OR REPLACE PROCEDURE Proc_CalTotalPrice() AS BEGIN UPDATE Orders SET Totalprice = (SELECT SUM(Extendedprice * (1 - Discount)*(1 + Tax)) FROM Lineitem WHERE Orders.Orderkey = Lineitem.Orderkey); END;</pre> ② 执行存储过程 Proc_CalTotalPrice()。			

续表

```
CALL Proc_CalTotalPrice();
```

(2) 有参数的存储过程

- ① 定义一个存储过程，更新给定订单的（含税折扣价）总价。

```
CREATE OR REPLACE PROCEDURE Proc_CalTotalPrice4Order(okey INTEGER)
AS
BEGIN
    UPDATE Orders SET Totalprice =
        (SELECT SUM(Extendedprice * (1 - Discount)*(1 + Tax))
         FROM Lineitem
         WHERE Orders.Orderkey = Lineitem.Orderkey AND
              Lineitem.Orderkey = okey);
END;
```

- ② 执行存储过程。

```
CALL Proc_CalTotalPrice4Order(5365);    /*带参数的调用*/
```

(3) 有局部变量的存储过程

- ① 定义一个存储过程，更新某个顾客的所有订单的（含税折扣价）总价。

```
CREATE OR REPLACE PROCEDURE Proc_CalTotalPrice4Customer(p_custname CHAR(25))
AS
DECLARE
    L_custkey INTEGER;
BEGIN
    SELECT Custkey INTO L_custkey    /*查找给定客户名对应的客户编号*/
    FROM Customer
    WHERE Name = TRIM(p_custname);
    /*TRIM 是系统函数，截去字符串前后空格*/
    UPDATE Orders SET Totalprice = /*修改指定客户编号的所有订单的总价*/
        (SELECT SUM(Extendedprice * (1 - Discount)*(1 + Tax))
         FROM Lineitem
         WHERE Orders.Orderkey = Lineitem.Orderkey AND
              Orders.Custkey = L_custkey);
END;
```

- ② 执行存储过程 Proc_CalTotalPrice4Customer()。

```
CALL Proc_CalTotalPrice4Customer('陈楷丰');
```


续表

③ 查看存储过程执行结果。

```
SELECT * FROM Orders
WHERE Custkey = (SELECT Custkey FROM Customer WHERE Name = '陈楷丰');
```

(4) 有输出参数的存储过程

① 定义一个存储过程，更新某个顾客的所有订单的（含税折扣价）总价。

```
CREATE OR REPLACE PROCEDURE Proc_CalTotalPrice4Customer2(p_custname
    CHAR(25), OUT p_totalprice REAL)
AS
DECLARE
    L_custkey INTEGER;
BEGIN
    SELECT Custkey INTO L_custkey          /*查找给定客户名对应的客户编号*/
    FROM Customer
    WHERE Name = TRIM(p_custname);
    RAISE NOTICE 'Custkey is %', L_custkey; /*提示客户编号信息*/

    UPDATE Orders SET Totalprice =          /*修改指定客户编号的所有订单的总价*/
    (SELECT SUM(Extendedprice * (1 - Discount) * (1 + Tax))
     FROM Lineitem
     WHERE Orders.Orderkey =
     Lineitem.Orderkey AND Orders.Custkey = L_custkey);
    /*查找指定客户编号的所有订单的总价的和，并通过输出参数输出该值*/
    SELECT SUM(Totalprice) INTO p_totalprice
    FROM Orders WHERE Custkey = L_custkey;
END;
```

② 执行存储过程 Proc_CalTotalPrice4Customer2()。

/*该存储过程的第二个参数为输出参数，在调用时给输出参数赋予 NULL，生成临时表，以结果集的形式显示出来*/

```
CALL Proc_CalTotalPrice4Customer2('陈楷丰',null);
```

③ 查看存储过程执行结果。

```
/*检查下列 SQL 语句执行结果与上述存储过程执行结果是否一致*/
SELECT SUM(Totalprice)
FROM Orders
WHERE Custkey = (SELECT Custkey
FROM Customer
```

续表

```
WHERE Name = '陈楷丰');
```

(5) 修改存储过程

- ① 修改存储过程名 Proc_CalTotalPrice4Order 为 CalTotalPrice4Order。

```
ALTER PROCEDURE Proc_CalTotalPrice4Order RENAME TO CalTotalPrice4Order;
```

- ② 编译存储过程 CalTotalPrice4Order。

```
ALTER PROCEDURE CalTotalPrice4Order(okey INTEGER) COMPILE;
```

(6) 删除存储过程

删除存储过程 CalTotalPrice4Order。

```
DROP PROCEDURE CalTotalPrice4Order;
```

实验总结:

存储过程、用户自定义函数可以通过 CALL 和 SELECT 语句调用。需要说明的是:

① 存储过程、用户自定义函数如果带有 OUT 或 INOUT 参数,则参数对应位置在调用时必须使用 NULL 或其他常量占位;运行所得是一个由一条或多条记录组成的结果集,每条记录中字段的顺序是 OUT 或 INOUT 参数对应的字段在前,最后返回 RETURN 语句对应的字段。

② SELECT 调用,就是执行普通的 SELECT 语句。对于存储过程,不能和其他任何常量、函数、存储过程等一并构成表达式使用,只能单独作为一个表达式出现在 SELECT 语句中。对于用户自定义函数,如果没有 OUT 或 INOUT 参数,则可以和其他常量、变量、对象名(如字段名)等组合成表达式使用。带有 OUT 或 INOUT 参数的函数不可以参与表达式的计算。

3. 思考题

- ① 试总结几种调试存储过程的方法。
- ② 存储过程中的 SELECT 语句与普通的 SELECT 语句格式有何不同? 执行方法有何不同?

实验 8.2 自定义函数实验

1. 实验内容和要求

掌握数据库编程语言 PL/SQL 以及数据库自定义函数的设计和使用方法,包括自定义函数定义、自定义函数运行、自定义函数更名、自定义函数删除、自定义函数的参数传递。掌握 PL/SQL

编程规范，规范设计自定义函数。

实验重点：自定义函数的定义和运行。

实验难点：自定义函数的参数传递方法。

2. 实验报告示例

实验报告

题目：自定义函数实验

姓名

日期

(1) 无参数的自定义函数

① 定义一个自定义函数，更新所有订单的（含税折扣价）总价，并返回所有订单的总价之和。

/*该自定义函数与实验 8.1 中的 Proc_CalTotalPrice()存储过程类似，区别在于该自定义函数具有一个 REAL 类型的返回值*/

CREATE OR REPLACE FUNCTION FUN_CalTotalPrice()

RETURN REAL

AS

DECLARE

res REAL; /* 自定义函数的返回值*/

BEGIN

UPDATE Orders SET Totalprice = /*更新所有订单的含税折扣价总价*/

(SELECT SUM(Extendedprice * (1-Discount)*(1 + Tax))

FROM Lineitem

WHERE Orders.Orderkey = Lineitem.Orderkey);

SELECT SUM(Totalprice) INTO res /*计算所有订单的含税折扣价总价之和*/

FROM Orders;

RETURN res; /*返回总价之和*/

END;

② 执行自定义函数 FUN_CalTotalPrice()。

SELECT FUN_CalTotalPrice();

/* 执行自定义函数，其返回值以结果集的方式返回和显示*/

(2) 有参数的自定义函数

① 定义一个自定义函数，更新并返回给定订单的总价。

CREATE OR REPLACE FUNCTION FUN_CalTotalPrice4Order(p_okey INTEGER)

RETURN REAL

AS

DECLARE

res REAL;

续表

```

BEGIN
    UPDATE Orders SET Totalprice=
        /*更新给定编号的订单的含税折扣价总价*/
        (SELECT SUM(Extendedprice * (1 - Discount)*(1 + Tax))
        FROM Lineitem
        WHERE Orders.Orderkey = Lineitem.Orderkey AND Lineitem.Orderkey=p_okey);

    SELECT Totalprice INTO res          /*查找给定订单的总价*/
    FROM Orders;
    WHERE Orderkey = p_okey;
    RETURN res;                        /*返回给定订单的总价*/
END;

```

② 执行自定义函数 FUN_CalTotalPrice4Order()。

```

/*更新并返回 5365 号订单的总价*/
CALL FUN_CalTotalPrice4Order(5365);

```

(3) 有局部变量的自定义函数

① 定义一个自定义函数，计算并返回某个顾客的所有订单的总价。

```

CREATE OR REPLACE FUNCTION FUN_CalTotalPrice4Customer(p_custname CHAR(25))
RETURN REAL
AS
DECLARE
    L_custkey INTEGER;          /*局部变量 L_custkey */
    res REAL;
BEGIN
    SELECT Custkey INTO L_custkey    /* 查找给定客户名的客户编号 */
    FROM Customer
    WHERE Name = TRIM(p_custname);

    RAISE NOTICE ' Custkey is %', L_custkey;    /* 提示客户编号信息 */

    /* 更新指定客户编号的所有订单的含税折扣价总价 */
    UPDATE Orders SET Totalprice =
        (SELECT SUM(Extendedprice * (1 - Discount)*(1 + Tax))
        FROM Lineitem
        WHERE Orders.Orderkey = Lineitem.Orderkey AND
            Orders.Custkey = L_custkey);

```

续表

```
/* 计算指定客户编号的所有订单的含税折扣价总价之和 */
SELECT SUM(Totalprice) INTO res
FROM Orders
WHERE Custkey = L_custkey;
RETURN res; /*返回总价之和*/
END;
```

② 执行自定义函数 FUN_CalTotalPrice4Customer()。

```
SELECT FUN_CalTotalPrice4Customer('陈楷丰');
```

(4) 有输出参数的自定义函数

① 定义一个自定义函数，计算并返回某个顾客的所有订单的总价。

/* 该函数定义一个输入参数 p_custname、一个输出参数 p_totalprice，还有一个返回值类型 REAL。通过输出参数的定义，该函数可以返回两个或者两个以上的值。该函数与 FUN_CalTotalPrice4Customer()基本类似，区别只在于该函数多了一个输出参数*/

```
CREATE OR REPLACE FUNCTION FUN_CalTotalPrice4Customer2(p_custname CHAR(25),
OUT p_totalprice REAL)
RETURN REAL
AS
DECLARE
    L_custkey INTEGER;
    res REAL;
BEGIN
    SELECT Custkey INTO L_custkey
    FROM Customer
    WHERE Name = TRIM(p_custname);

    RAISE NOTICE ' Custkey is %', L_custkey;

    UPDATE Orders SET Totalprice =
        (SELECT SUM(Extendedprice * (1 - Discount)*(1 + Tax))
        FROM Lineitem
        WHERE Orders.Orderkey = Lineitem.Orderkey AND
        Orders.Custkey = L_custkey);

    SELECT SUM(Totalprice) INTO p_totalprice
    FROM Orders
    WHERE Custkey = L_custkey;
```


续表

```
Res := p_totalprice;  
RETURN res;  
END;
```

- ② 执行自定义函数 FUN_CalTotalPrice4Customer2()。

```
SELECT FUN_CalTotalPrice4Customer2('陈楷丰',null);
```

(5) 修改自定义函数

- ① 修改自定义函数名 FUN_CalTotalPrice4Order 为 CalTotalPrice4Order。

```
ALTER FUNCTION FUN_CalTotalPrice4Order RENAME TO CalTotalPrice4Order;
```

- ② 编译自定义函数 CalTotalPrice4Order。

```
ALTER FUNCTION CalTotalPrice4Order(okey INTEGER) COMPILE;
```

(6) 删除自定义函数

删除自定义函数 CalTotalPrice4Order。

```
DROP FUNCTION CalTotalPrice4Order;
```

实验总结:

- ① 自定义函数也称为存储函数。自定义函数和存储过程类似，都是持久性存储模块，不同之处在于函数必须指定返回值的类型。
- ② 使用存储过程和自定义函数等方法，用户可以自定义程序逻辑，开发完成业务逻辑复杂的应用系统。

3. 思考题

- (1) 试分析自定义函数与存储过程的区别与联系。
- (2) 如何让自定义函数可以返回多个值？如何使用该函数？

实验 8.3 游标实验

1. 实验内容和要求

掌握 PL/SQL 游标的设计、定义和使用方法，理解 PL/SQL 游标按行操作和 SQL 按结果集操作的区别与联系；掌握各种类型游标的特点、区别与联系。

实验重点：游标的定义和使用。

实验难点：游标类型。

2. 实验报告示例

实验报告

题目: 游标实验

姓名

日期

(1) 普通游标

- ① 定义一个存储过程, 用游标实现计算所有订单的总价。

```

CREATE OR REPLACE PROCEDURE ProcCursor_CalTotalPrice()
AS /*定义存储过程, 循环计算和更新每个订单的总价 totalprice*/
DECLARE
    L_orderkey INTEGER;
    L_totalprice REAL;
    CURSOR mycursor FOR /*定义下面 SELECT 语句的游标*/
        SELECT Orderkey, Totalprice
        FROM Orders;
BEGIN
    OPEN mycursor; /* 打开游标 */
    LOOP /*对每个订单记录循环, 修改其 Totalprice 属性值*/
        FETCH mycursor INTO L_orderkey, L_totalprice; /*获取游标当前记录*/
        IF mycursor%NOTFOUND THEN
            EXIT; /*游标找不到记录, 则退出*/
        END IF;
        /*计算当前订单所有明细项目的含税折扣价格总和*/
        SELECT SUM(Extendedprice * (1 - Discount)*(1 + Tax)) INTO L_totalprice
        FROM Lineitem
        WHERE Orderkey = L_orderkey; /*修改当前订单的 Totalprice 属性值*/
        UPDATE Orders SET Totalprice = L_totalprice
        WHERE Orderkey = L_orderkey;
    END LOOP;
    CLOSE mycursor; /* 关闭游标 */
END;

```

- ② 执行存储过程 ProcCursor_CalTotalPrice()。

```
CALL ProcCursor_CalTotalPrice();
```

(2) REFCURSOR 类型游标

- ① 定义一个存储过程, 用游标实现计算所有订单的总价。

```

/*该存储过程与 ProcCursor_CalTotalPrice()存储过程类似, 唯一区别就是所定义的游标类型不同 */
CREATE OR REPLACE PROCEDURE ProcRefCursor_CalTotalPrice()
AS

```

续表

```

DECLARE
    L_orderkey INTEGER;
    L_totalprice REAL;
    mycursor REFCURSOR;          /*定义 REFCURSOR 类型游标*/
BEGIN
    /*打开 REFCURSOR 类型游标, 游标记录集为 Orders 订单表所有记录*/
    OPEN mycursor FOR SELECT Orderkey, Totalprice
    FROM Orders;
    LOOP
        FETCH mycursor INTO L_orderkey, L_totalprice;
        IF mycursor%NOTFOUND THEN
            EXIT;
        END IF;
        SELECT SUM(Extendedprice * (1 - Discount)*(1 + Tax)) INTO L_totalprice
        FROM Lineitem
        WHERE Orderkey = L_orderkey;

        UPDATE Orders SET Totalprice = L_totalprice
        WHERE Orderkey = L_orderkey;
    END LOOP;
    CLOSE mycursor;
END;

```

② 执行存储过程 ProcRefCursor_CalTotalPrice()。

```
CALL ProcRefCursor_CalTotalPrice();
```

(3) 记录变量与游标

① 定义一个存储过程, 用游标实现计算所有订单的总价。

/*该存储过程与 ProcCursor_CalTotalPrice()存储过程类似, 唯一区别就是所定义的游标结果类型为记录类型的变量*/

```

CREATE OR REPLACE PROCEDURE ProcRecCursor_CalTotalPrice()
AS
DECLARE
    L_totalprice REAL;
    res RECORD;          /*定义游标结果为记录类型变量*/
    CURSOR mycursor FOR
        SELECT Orderkey, Totalprice
        FROM Orders;
BEGIN

```


续表

```

OPEN mycursor;
LOOP
    /*从游标中取出的结果保存在 res 记录类型的变量中*/
    FETCH mycursor INTO res;
    IF mycursor%NOTFOUND THEN EXIT;
    END IF;
    /*从记录变量 res 中获得当前订单的订单编号, 计算当前订单的所有明细项目的
    含税折扣价总和*/
    SELECT SUM(Extendedprice * (1 - Discount)*(1 + Tax)) INTO L_totalprice
    FROM Lineitem
    WHERE Orderkey = res.Orderkey;

    UPDATE Orders SET Totalprice = L_totalprice
    WHERE Orderkey =res.Orderkey;
END LOOP;
CLOSE mycursor;
END;

```

② 执行存储过程 ProcRecCursor_CalTotalPrice()。

```
CALL ProcRecCursor_CalTotalPrice();
```

(4) 带参数的游标

① 定义一个存储过程, 用游标实现计算指定国家的用户订单的总价。

/*该存储过程与 ProcCursor_CalTotalPrice()存储过程类似, 区别在于该存储过程带有一个参数, 所定义的游标为带参数的游标, 游标结果定义为记录类型*/

```

CREATE OR REPLACE PROCEDURE ProcParaCursor_CalTotalPrice(p_nationname CHAR(20))
AS
DECLARE
    L_totalprice REAL;
    res RECORD;          /*定义游标结果为记录类型变量*/
    /*定义一个带参数的游标, 查询指定国家名称的顾客的订单及其总价*/
    CURSOR mycursor(c_nationname CHAR(25)) FOR
        SELECT O.Orderkey, O.Totalprice
        FROM Orders O, Customer C, Nation N
        WHERE O.Custkey = C.Custkey AND C.Nationkey = N.Nationkey AND
        TRIM(N.Name) = TRIM(c_nationname);
BEGIN
    /*打开游标: 把存储过程的参数 p_nationname 传递给游标的参数 c_nationname*/
    OPEN mycursor(p_nationname);
    LOOP

```

续表

```
FETCH mycursor INTO res;  
IF mycursor% NOTFOUND THEN  
    EXIT;  
END IF;  
SELECT SUM(Extendedprice*(1-Discount)*(1+Tax)) INTO L_totalprice  
FROM Lineitem  
WHERE Orderkey = res.Orderkey;  
  
UPDATE Orders SET Totalprice = L_totalprice  
WHERE Orderkey =res.Orderkey;  
END LOOP;  
CLOSE mycursor;  
END;
```

② 执行存储过程 ProcParaCursor_CalTotalPrice()。

```
CALL ProcParaCursor_CalTotalPrice('中国');    /*计算中国用户订单的总价*/
```

实验总结:

① REFCURSOR 类型的游标定义一个游标引用变量,只是在打开该类型游标时才指定具体的 SELECT 语句,以便产生游标的结果集。因此,REFCURSOR 实质上是定义了一个动态游标,可以灵活方便地根据程序运行时情况动态设置游标的 SELECT 查询结果集。

② 从任务一可以看出,游标可以实现对数据库记录的逐条处理,而不是整个结果集一起处理,因此游标是在 PL/SQL 中实现过程化处理的核心功能。

③ 从任务三看出,记录变量对于游标结果记录的处理很方便,通过记录变量可以直接访问记录的每个属性,而无须为记录的每个属性定义相应的变量。

④ PL/SQL 中 %TYPE、%ROWTYPE 和 RECORD 等数据类型的区别与联系如下:

a. %TYPE 提供一个表字段数据类型的变量。例如,在 users 表中有字段 User_id,要声明一个和 users.User_id 类型相同的变量,可以写为 user_id users.User_id%TYPE。缺点是:使用 %TYPE,必须要知道所引用的结构的数据类型。优点是:即使被引用项的表字段数据类型发生了变化,也不需要修改 PL/SQL 的过程体定义。

b. %ROWTYPE 提供一个复合类型变量,称为行变量。这样的变量可以保存一次 SELECT 命令结果的完整一行,只要命令的字段集匹配该变量声明的类型。可将一个行变量声明为与一个现有的表或者视图的行类型相同,方法是使用 table_name%ROWTYPE 表示。在一个行类型的变量中,只能访问用户定义的表中行的属性。

c. RECORD 提供一个记录变量,类似于行类型变量,但是它们没有预定义的结构,而是在 SELECT 命令中获取实际的行结构,这点和 Oracle 不同。在 KingbaseES 的 PL/SQL 过程体中,正是这种预先没有确定结构的特性,为用户提供了极大的灵活性。

3. 思考题

- ① 试分析说明 REFCURSOR 类型游标的优点。
- ② 分析游标是如何架起“数据库按集合操作数据”和“过程语言按记录操作数据”之间的桥梁的。

实验 8.4 基于 JDBC 的数据库应用开发实验

1. 实验内容和要求

掌握基于 JDBC 驱动的数据库应用开发方法、基于 JDBC 驱动的数据库连接方法，实现数据库数据操纵等应用开发常见功能。

实验重点：基于 JDBC 驱动的数据库连接方法、数据库数据操纵功能等。

实验难点：不同的数据库应用开发工具包含不同的开发框架和模式，以及如何较为熟练地使用所选择的应用开发工具。

2. 实验报告示例

实验报告				
题目：基于 JDBC 的数据库应用开发实验	姓名		日期	
在本实验中，以 Eclipse 开发环境和 KingbaseES 数据库为例实现一个完整的示例程序，其源代码 JDBCTest.java 附在思考题之后。该程序实现的功能为：把 KingbaseES 的 Samples 数据库中 S-C 模式下的 Student 表数据复制到 SQL SERVER 的 SamplesBackup 数据库中相同模式的相同表中。为简单起见，也可以在 KingbaseES 中创建 SameplesBackup 数据库，建立 S-C 模式和 Student 表，然后实现上述复制功能。该程序的框架如下： (1) 基于 JDBC 驱动的数据库连接方法 <pre>/* Step 1: 定义句柄和变量 */ /* Step 2: 初始化环境 */ /* Step 3: 建立连接 */</pre> (2) 基于 JDBC 驱动的数据库数据操纵方法 <pre>/* Step 4: 初始化语句句柄 */ /* Step 5: 两种方式执行语句 */ /* 预编译带有参数的语句 */ /* 直接执行 SQL 语句*/ /* Step 6: 处理结果集并执行预编译后的语句*/</pre> (3) 中止基于 JDBC 驱动的数据库连接 <pre>/* Step 7: 中止处理*/</pre>				

续表

实验总结:

Java 是一种面向对象的高级编程语言, 直接使用 JDBC 可以设计访问不同数据源之间数据库的应用程序; 但是在实际的数据库应用系统开发中通常并不直接使用 JDBC 访问数据库, 而是使用 Hibernate、MyBatis 等数据库中间件来访问数据库。

3. 思考题

① 试修改源程序 JDBCTest.java, 实现零件供应销售数据库模式数据库 Sales 模式下所有表的复制。

② 请调查目前比较流行的软件开发环境, 分析它们在基于 JDBC 驱动开发数据库应用方面各有哪些优缺点。

Eclipse 环境中编程实现访问数据库的源程序 JDBCTest.java 如下:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.Properties;

public class JDBCTest {
    public static void main(String[] args) {

        /* Step 1 定义句柄和变量 */
        String sSno, sName, sSex, sDepart;
        Integer sAge;

        /* Step 2 初始化环境 */
        /* Step 3 :建立连接 */

        String driverName = "com.kingbase.Driver";           //连接源数据库
        String src_url = "jdbc:kingbase://localhost:54321/SAMPLES";
        Properties src_info = new Properties();
        src_info.put ("user", "SYSTEM");
        src_info.put ("password", "MANAGER");
        src_info.put ("EscapeProcessing", "false");

        String dst_url = "jdbc:kingbase://localhost:54321/SameplesBackup";
        //连接目标数据库
        Properties dst_info = new Properties();
```



```
dst_info.put ("user", "SYSTEM");
dst_info.put ("password", "MANAGER");
dst_info.put ("EscapeProcessing", "false");

try{
    Class.forName(driverName);
    Connection src_conn = DriverManager.getConnection(src_url,src_info);
    Connection dst_conn = DriverManager.getConnection(dst_url, dst_info);

    /* Step 4 :初始化语句句柄 */
    Statement src_stmt = src_conn.createStatement();
    PreparedStatement dst_prestmt = dst_conn.prepareStatement("INSERT INTO
        \'S-C\'.STUDENT (SNO,SNAME,SSEX,SAGE,SDEPT) VALUES (?, ?, ?, ?, ?)");
    /* Step 5 :两种方式执行语句 */
    /*预编译带有参数的语句*/
    /*直接执行 SQL 语句*/
    ResultSet rs = src_stmt.executeQuery("SELECT * FROM \'S-C\'.Student");

    /* Step 6: 处理结果集并执行预编译后的语句*/
    while (rs.next())
    {
        sSno = rs.getString("SNO");
        sName = rs.getString("SNAME");
        sSex = rs.getString("SSEX");
        sAge = rs.getInt("SAGE");
        sDepart = rs.getString("SDEPT");

        dst_prestmt.setString(1, sSno);
        dst_prestmt.setString(2, sName);
        dst_prestmt.setString(3, sSex);
        dst_prestmt.setInt(4, sAge);
        dst_prestmt.setString(5, sName);

        dst_prestmt.executeUpdate();

        System.out.println(sSno + sName + sSex + sDepart + sAge);
    }
    /* Step 7 终止处理*/
    rs.close();
    src_stmt.close();
    dst_prestmt.close();
}
```

```

        src_conn.close();
        dst_conn.close();
    }
    catch (Exception e)
    {
        System.out.println(e.getMessage());
    }
}
}

```

实验 8.5 数据库大作业

1. 实验内容和要求

掌握综合运用数据库原理、方法和技术，进行数据库应用系统分析、设计和开发的能力。为某个部门或单位开发一个数据库应用系统，具体内容包括：对某个部门或单位的业务和数据进行调查，进行系统分析、系统设计、数据库设计、数据库创建和数据加载；进行数据库应用软件开发、系统测试、系统分析设计和开发文档撰写；进行软件、文档和数据库提交，数据库应用系统运行演示和大作业汇报。

能够针对某个部门或单位的应用需求，通过系统分析，从数据库数据和应用系统功能两方面进行综合设计，实现一个完整的数据库应用系统，同时培养团队合作精神。要求 5~6 位同学组成一个开发小组，每位同学承担不同的角色（例如项目管理员、DBA、系统分析员、系统设计员、系统开发员、系统测试员）。撰写系统设计和开发文档，提交系统文档、数据库应用软件和数据库。每个小组进行 30 分钟的报告和答辩，讲解设计方案，演示系统运行，汇报分工与合作情况。

实验重点：数据库设计与数据库应用软件开发。

实验难点：综合运用系统分析与设计方法，从数据和功能两方面协调设计一个完整的数据库应用系统。要能熟练掌握和运用一种主流 DBMS 应用开发工具，进行数据库应用软件开发。

1. 实验报告示例

实验报告			
题目：数据库大作业	姓名		日期
在“数据库系统概论”精品课程网站上有同学们完成的一些大作业实验报告示例，可供读者参考学习。 数据库设计和应用开发步骤主要包括： ① 系统调查 ② 系统分析 ③ 系统设计			

续表

- ④ 数据库设计
- ⑤ 数据库创建和数据加载
- ⑥ 数据库应用软件的功能设计和开发
- ⑦ 数据库应用系统测试
- ⑧ 分析设计和开发文档撰写
- ⑨ 应用软件、数据库和文档提交
- ⑩ 应用系统演示和汇报

实验总结:

① 数据库大作业对学生的要求比较高,不仅要求学生掌握数据库基本原理和方法,数据库设计方法,还需要掌握 Java 或 Python 等数据库开发语言、工具和相应的 Web 开发框架等知识。

② 在数据库课程教学开始就布置该项数据库大作业实验,讲清楚实验内容和要求,以便学生提前准备,学习和熟悉相关的开发语言和工具,这将有利于本实验的完成。

3. 思考题

- ① 数据库应用系统的功能设计对数据库设计有何影响?
- ② 如何协调数据库应用系统的功能设计与数据库设计?
- ③ 在数据库应用开发中,使用的程序设计语言的数据类型与数据库的数据类型如何相互转换?

第10章实验 数据库性能监视与调优

数据库性能监视与调优实验分为三个选修实验项目(见表9),它们都属于验证性与设计性相结合的实验。

表9 “数据库性能监视与调优”实验项目一览表

序号	实验项目名称	开设类别	难易程度	建议实验学时	对应《概论》章节
实验 10.1	数据库查询性能调优实验	选修	难	4	第10章
*实验 10.2	数据库性能监视实验	选修	难	4	第10章
*实验 10.3	数据库系统配置参数调优实验	选修	难	4	第10章

实验 10.1 数据库查询性能调优实验

1. 实验内容和要求

理解和掌握数据库查询性能调优的基本原理和方法。学会使用 EXPLAIN 命令分析查询执